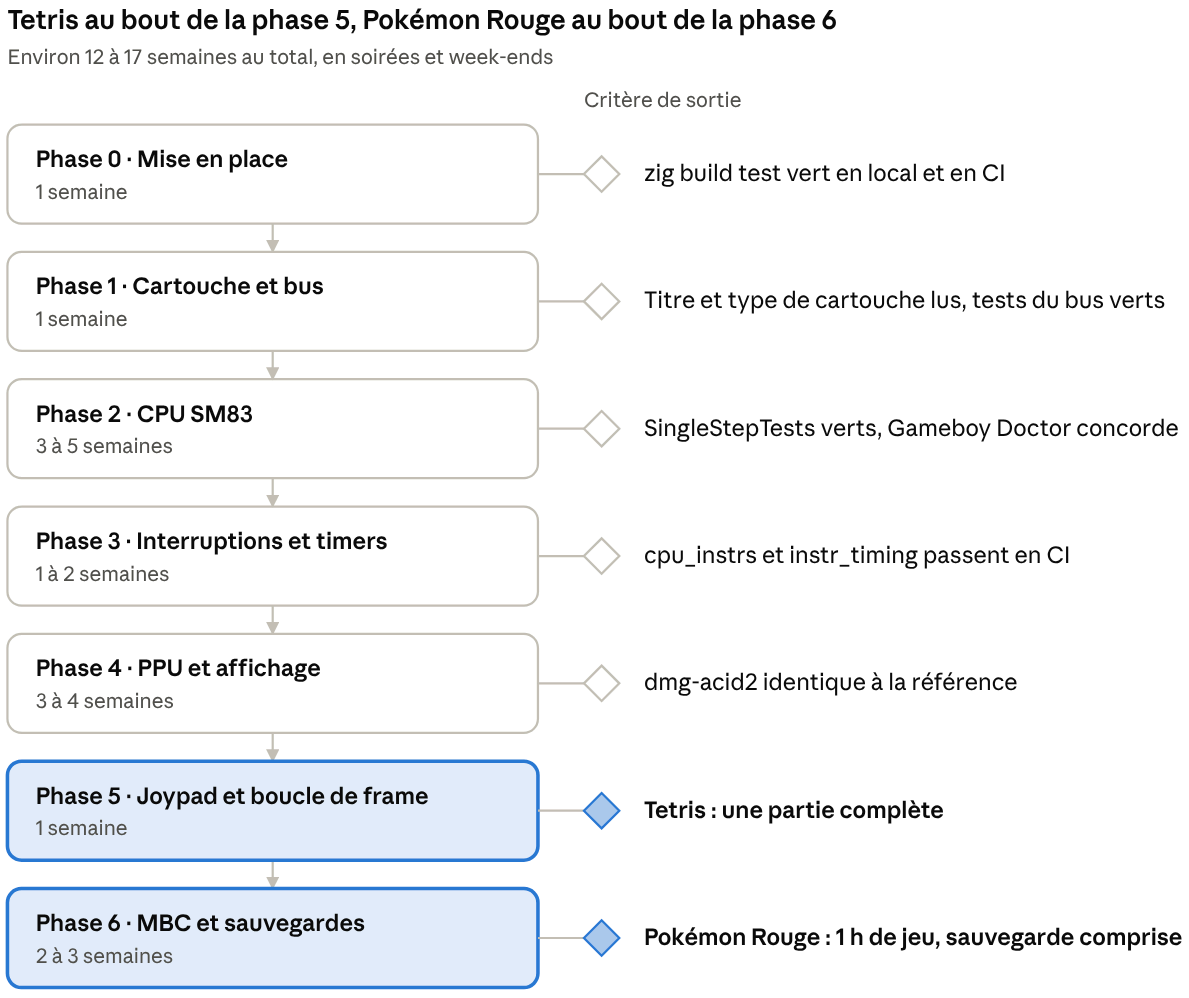
Émulateur Game Boy en Zig — Feuille de route

Nov 4, 2026 · @Saussice

Vue d'ensemble

Sept phases mènent de `git init` à une partie sauvegardée de Pokémon Rouge, en 12 à 17 semaines environ à temps partiel. Chaque phase se termine par un critère de sortie vérifiable, autant que possible par un test automatique.



feuille de route · 7 phases, un critère de sortie chacune

Deux principes tiennent tout le projet. Le cœur de l'émulateur ne dépend jamais de l'affichage, donc tout se teste sans fenêtre. Et chaque composant a sa ROM de test avant d'être écrit : SingleStepTests pour le CPU, Blargg pour les timers, dmg-acid2 pour le PPU,

mooneye pour les MBC.

Phase 0 — Mise en place (1 semaine)

Objectif : un dépôt qui compile, lance ses tests en local et en CI, avec un cœur d'émulateur séparé de l'affichage dès le premier jour.

Échauffement Zig. Fais une bonne partie de Ziglings avant d'écrire une ligne d'émulateur. Vise surtout les entiers, `@truncate` / `@intCast`, les opérateurs `+%` et `-%`, les `packed struct`, les `union(enum)`, `comptime` et les blocs `test`.

Version. Installe la dernière version stable depuis ziglang.org et épingle-la : note-la dans le README et dans `minimum_zig_version` de `build.zig.zon`. Ne change de version qu'entre deux phases, jamais en plein milieu.

Dépôt.

- [ ] `git init`, puis `zig init` pour obtenir `build.zig`, `build.zig.zon` et `src/`
- [ ] `.gitignore` : `.zig-cache/`, `zig-out/`, `roms/`, `*.sav`
- [ ] Ne jamais committer de ROM commerciale. Les ROMs de test vont dans `test-roms/`, téléchargées par un petit script plutôt que committées
- [ ] README : objectif, version de Zig, commandes `zig build run -- <rom>` et `zig build test`

Structure cible. Le cœur (`src/gb/`) ne dépend d'aucune bibliothèque graphique. C'est ce qui le rend testable sans fenêtre, et portable plus tard vers WebAssembly.

```
src/
  main.zig            // point d'entrée, arguments
  gb/
    gameboy.zig       // assemble tout, step() et runFrame()
    cpu.zig
    bus.zig
    cartridge.zig     // en-tête + MBC
    interrupts.zig
    timer.zig
    ppu.zig
    joypad.zig
  frontend/
    window.zig        // SDL ou raylib, ajouté en phase 4
tests/
  blargg.zig          // harnais ROMs de test
```

build.zig. Trois étapes : `run` (avec passage d'arguments), `test` (tests unitaires et harnais ROMs) et un module `gb` exporté que l'exécutable et les tests importent.

CI. Un workflow GitHub Actions qui installe la même version de Zig (par exemple avec l'action `mlugg/setup-zig`) puis lance `zig build test` à chaque push.

Critère de sortie : `zig build test` passe en local et en CI sur un test trivial, et `zig build run -- fichier.gb` affiche la taille du fichier.

Phase 1 — Cartouche et bus mémoire (1 semaine)

Objectif : charger une ROM, lire son en-tête et router chaque adresse 16 bits vers le bon composant.

Cartouche. Lis le fichier dans un tableau d'octets et parse l'en-tête :

Adresse	Contenu
0x0134–0x0143	Titre
0x0147	Type de cartouche (0x00 = ROM seule, 0x01–0x03 = MBC1, 0x0F–0x13 = MBC3)
0x0148	Taille de la ROM
0x0149	Taille de la RAM externe

Adresse	Contenu
0x014D	Checksum de l'en-tête

Modélise la cartouche comme une `union(enum)` (`rom_only`, `mbc1`, `mbc3`…) : le `switch` exhaustif t'obligera à traiter chaque type. Seul `rom_only` est implémenté pour l'instant ; les autres renvoient une erreur explicite.

Carte mémoire. Deux fonctions `read(addr: u16) u8` et `write(addr: u16, value: u8)` qui routent :

Plage	Zone
0x0000–0x7FFF	ROM (banque 0, puis banque commutable)
0x8000–0x9FFF	VRAM
0xA000–0xBFFF	RAM de la cartouche
0xC000–0xDFFF	WRAM
0xE000–0xFDFF	Miroir de 0xC000–0xDDFF (echo RAM)
0xFE00–0xFE9F	OAM (sprites)
0xFEA0–0xFEFF	Inutilisable (lit 0xFF)
0xFF00–0xFF7F	Registres d'E/S
0xFF80–0xFFFE	HRAM
0xFFFF	IE (interruptions activées)

Les registres d'E/S non encore implémentés renvoient 0xFF et ignorent les écritures. Garde une trace (log ou compteur) des accès non gérés : c'est ta liste de travail pour la suite.

Tests.

- [ ] Le checksum d'en-tête calculé correspond à 0x014D sur toutes les ROMs de test
- [ ] Une écriture en 0xC010 se relit en 0xE010
- [ ] Une écriture dans la zone ROM d'une cartouche `rom_only` ne change rien

Critère de sortie : `zig build run -- tetris.gb` affiche le titre et le type de cartouche, et les tests du bus passent.

Phase 2 — CPU SM83 (3 à 5 semaines)

Objectif : les 245 opcodes valides de la table principale et les 256 opcodes préfixés `CB`, chacun avec son nombre de cycles exact.

Registres. Huit registres 8 bits (A, F, B, C, D, E, H, L), plus SP et PC sur 16 bits. F est un `packed struct(u8)` dont les 4 bits bas valent toujours 0. Ajoute des accesseurs pour les paires `af`, `bc`, `de` et `hl` ; `setAf` doit forcer ces 4 bits à 0.

État de départ. Saute la ROM de boot et initialise directement l'état post-boot d'une DMG : AF=0x01B0, BC=0x0013, DE=0x00D8, HL=0x014D, SP=0xFFFE, PC=0x0100.

Décodage. Découpe chaque opcode en trois champs, `x: u2` (bits 7–6), `y: u3` (bits 5–3) et `z: u3` (bits 2–0). La valeur 0–7 de `y` ou `z` désigne un registre dans l'ordre B, C, D, E, H, L, (HL), A. Branche ensuite `switch (opcode) { inline else => |op| ... }` : chaque branche reçoit `op` connu à la compilation et le décodage disparaît du code généré.

Ordre d'implémentation conseillé.

1. `NOP`, chargements 8 bits (`LD r,r'`, `LD r,n`, `LD (HL),r`)
2. Chargements 16 bits, `PUSH` et `POP`
3. ALU 8 bits : `ADD`, `ADC`, `SUB`, `SBC`, `AND`, `XOR`, `OR`, `CP`, `INC`, `DEC`
4. ALU 16 bits : `ADD HL,rr`, `INC rr`, `DEC rr`, `ADD SP,e`, `LD HL,SP+e`
5. Sauts et appels : `JP`, `JR`, `CALL`, `RET`, `RETI`, `RST`, versions conditionnelles comprises
6. Rotations sur A (`RLCA`, `RRCA`, `RLA`, `RRA`), puis tout le préfixe `CB` (rotations, décalages, `SWAP`, `BIT`, `RES`, `SET`)
7. Divers : `DAA`, `CPL`, `SCF`, `CCF`, `DI`, `EI`, `HALT`, `STOP`

Cycles. Chaque instruction renvoie son nombre de M-cycles (1 M-cycle = 4 cycles d'horloge). Les sauts conditionnels coûtent plus cher quand ils sont pris. Ce compteur pilotera les timers et le PPU dès la phase 3.

Tests, du plus fin au plus large.

- [ ] Tests unitaires écrits à la main pour chaque famille : surtout les flags Z, N, H, C
- [ ] Les SingleStepTests sm83 : des milliers de cas JSON par opcode (état avant, état après). Un harnais avec `std.json` les exécute tous. C'est le meilleur outil TDD pour le CPU
- [ ] Gameboy Doctor : ton émulateur écrit une ligne d'état par instruction pendant `cpu_instrs`, et l'outil indique la première instruction divergente. Il faut faire lire 0x90 à LY (0xFF44) pour que les journaux concordent

Critère de sortie : tous les SingleStepTests passent, et le journal de Gameboy Doctor concorde sur les ROMs `cpu_instrs` qui n'ont pas besoin des interruptions.

Phase 3 — Interruptions, timers et harnais Blargg (1 à 2 semaines)

Objectif : `cpu_instrs` (les 11 ROMs) et `instr_timing` de Blargg passent automatiquement dans `zig build test`, sans fenêtre.

Port série, en premier. Quand le jeu écrit 0x81 dans SC (0xFF02), récupère l'octet de SB (0xFF01) et ajoute-le à un tampon. Les ROMs Blargg y écrivent leur résultat en texte.

Harnais. Une fonction qui charge une ROM, exécute jusqu'à voir `Passed` ou `Failed` dans le tampon série (ou jusqu'à un délai maximal), puis fait échouer le test avec la sortie complète. Un test Zig par ROM : tu sais immédiatement laquelle casse.

Interruptions.

- IE (0xFFFF) et IF (0xFF0F), plus le flag interne IME
- Vecteurs : VBlank 0x40, STAT 0x48, Timer 0x50, Série 0x58, Joypad 0x60 ; la priorité suit cet ordre
- Prise en charge : IME à 0, bit correspondant de IF effacé, PC empilé, saut au vecteur, 5 M-cycles
- `EI` n'active IME qu'après l'instruction suivante ; `DI` agit immédiatement
- `HALT` se réveille dès que `IE & IF != 0`, même si IME vaut 0 (dans ce cas sans appeler le vecteur)
- Le « HALT bug » : `HALT` avec IME à 0 et une interruption déjà en attente fait lire deux fois l'octet suivant

Timers.

- DIV (0xFF04) est l'octet haut d'un compteur interne 16 bits incrémenté à chaque cycle ; toute écriture le remet à 0
- TIMA (0xFF05) s'incrémente à la fréquence choisie par TAC (0xFF07) : 4096, 262144, 65536 ou 16384 Hz
- Au débordement, TIMA prend la valeur de TMA (0xFF06) et l'interruption Timer est demandée
- Dérive TIMA des bits du compteur interne plutôt que d'un compteur séparé : les cas limites deviennent justes gratuitement

Boucle principale. `step()` exécute une instruction (ou un cycle de HALT), récupère ses M-cycles et fait avancer le timer d'autant, puis traite les interruptions en attente.

Critère de sortie : les 11 ROMs `cpu_instrs` et `instr_timing` passent en CI.

Phase 4 — PPU et affichage (3 à 4 semaines)

Objectif : un framebuffer 160×144 correct, validé par dmg-acid2, puis affiché dans une fenêtre.

Machine à états. Une ligne dure 456 dots, une frame 154 lignes, soit 70 224 dots (17 556 M-cycles). Sur les lignes 0 à 143, le PPU enchaîne le mode 2 (scan de l'OAM, 80 dots), le mode 3 (dessin) et le mode 0 (HBlank). Les lignes 144 à 153 sont en mode 1 (VBlank), et l'interruption VBlank part à l'entrée de la ligne 144.

Registres. LCDC 0xFF40, STAT 0xFF41, SCY/SCX 0xFF42–0xFF43, LY 0xFF44, LYC 0xFF45, DMA 0xFF46, BGP 0xFF47, OBP0/OBP1 0xFF48–0xFF49, WY/WX 0xFF4A–0xFF4B.

Rendu par ligne. Dessine une ligne entière d'un coup au passage en mode 0. Pas besoin de FIFO de pixels : un rendu par ligne suffit pour Tetris, Pokémon et dmg-acid2. Ordre :

1. Fond : tuiles 2 bits par pixel (16 octets par tuile), tilemap et adressage des tuiles choisis par LCDC, défilement SCX/SCY, palette BGP
2. Fenêtre : position WX−7 / WY, avec un compteur de ligne interne qui n'avance que sur les lignes où la fenêtre est réellement dessinée
3. Sprites : 10 au maximum par ligne, tailles 8×8 et 8×16, retournements, palettes OBP0/OBP1, priorité derrière le fond, couleur 0 transparente

Le reste.

- [ ] DMA OAM : une écriture en 0xFF46 copie 160 octets vers 0xFE00
- [ ] Interruptions STAT (modes 0, 1, 2 et coïncidence LY=LYC) et flag de coïncidence
- [ ] Écran éteint (bit 7 de LCDC à 0) : LY reste à 0, rien n'est dessiné

Test headless d'abord. Écris une fonction qui exporte le framebuffer en image PPM, puis un test qui exécute dmg-acid2 quelques frames et compare au PNG de référence (converti une fois pour toutes). Tu restes en TDD même pour le graphisme.

Fenêtre. Ensuite seulement, branche un paquet Zig pour SDL3 ou raylib dans `build.zig.zon`, dans `src/frontend/`. Le frontend reçoit le framebuffer à chaque frame et l'envoie dans une texture mise à l'échelle (×3 ou ×4).

Critère de sortie : dmg-acid2 identique pixel pour pixel à la référence, et l'écran titre de Tetris s'affiche.

Phase 5 — Joypad et Tetris jouable (1 semaine)

Objectif : une partie de Tetris complète, à la bonne vitesse, au clavier.

Joypad (0xFF00). Le jeu écrit le bit 4 (croix directionnelle) ou le bit 5 (boutons) à 0 pour choisir le groupe à lire. Les bits 0 à 3 renvoient l'état du groupe, en logique inversée : 0 = appuyé. Les bits 6 et 7 se lisent toujours à 1. Un appui demande l'interruption Joypad.

Boucle de frame. `runFrame()` exécute 17 556 M-cycles, puis le frontend affiche l'image, lit le clavier et attend. La Game Boy tourne à environ 59,73 images par seconde : cale-toi sur cette cadence plutôt que sur la VSync de l'écran, ou le jeu ira trop vite sur un moniteur 144 Hz.

Touches. Flèches pour la croix, X et Z pour A et B, Entrée pour Start, Retour arrière pour Select. Ajoute une touche d'accélération (sans limite de cadence) : précieuse pour déboguer.

Vérifications.

- [ ] L'écran de copyright puis le titre s'affichent
- [ ] Le menu de choix du type de jeu répond aux touches
- [ ] Les pièces tombent, tournent et se posent
- [ ] Une ligne complète disparaît et le score augmente
- [ ] Le game over s'affiche et on peut relancer une partie

Critère de sortie : tu joues une partie entière sans bug visible. Premier vrai jalon du projet.

Phase 6 — MBC, sauvegardes et Pokémon Rouge (2 à 3 semaines)

Objectif : commencer une partie de Pokémon Rouge, sauvegarder en jeu, fermer l'émulateur et reprendre la partie.

Pokémon Rouge est une cartouche de type 0x13 (MBC3 + RAM + pile), sans horloge temps réel. Commence quand même par le MBC1, plus simple et couvert par les tests mooneye.

MBC1. Les écritures dans la zone ROM sont des commandes :

Plage d'écriture	Effet
0x0000–0x1FFF	Active la RAM si la valeur se termine par 0xA
0x2000–0x3FFF	Numéro de banque ROM, 5 bits (0 devient 1)
0x4000–0x5FFF	Banque RAM, ou bits hauts de la banque ROM
0x6000–0x7FFF	Mode de banking (simple ou avancé)

Valide-le avec les ROMs `emulator-only/mbc1` de mooneye.

MBC3. Même principe, avec un numéro de banque ROM sur 7 bits (0 devient 1) et jusqu'à 4 banques RAM sélectionnées par 0x00–0x03 en 0x4000–0x5FFF. Les valeurs 0x08–0x0C y sélectionnent les registres de l'horloge : comme Pokémon Rouge n'en a pas, un simple stub suffit pour l'instant.

Sauvegardes. Si la cartouche a une pile, charge `<rom>.sav` au démarrage s'il existe, et écris la RAM externe à la fermeture. Pour ne rien perdre en cas de plantage, écris aussi quelques secondes après la dernière modification de la RAM.

Vérifications dans Pokémon Rouge.

- [ ] L'intro (Ectoplasma contre Nidorino) et l'écran titre s'affichent correctement
- [ ] Les boîtes de dialogue (la fenêtre du PPU) s'affichent sans décalage
- [ ] Le déplacement dans Bourg Palette est fluide, les changements de carte fonctionnent
- [ ] Un combat complet fonctionne (animations, menus, barres de PV)
- [ ] Sauvegarde en jeu, fermeture, redémarrage, « Continuer »

Critère de sortie : une heure de jeu sans plantage, sauvegarde comprise. Objectif du projet atteint.

Après Pokémon — pistes bonus

Aucune n'est nécessaire à l'objectif ; choisis selon ce qui t'amuse.

- **Débogueur intégré** : pas à pas, points d'arrêt sur PC ou sur adresse mémoire, vue de la VRAM et des tilemaps. À construire tôt si les phases 4 à 6 deviennent pénibles à déboguer
- **Son (APU)** : quatre canaux (deux carrés, une onde, un bruit), mixés et envoyés au frontend. Gros morceau, très gratifiant

- **Précision** : passer les tests `acceptance` de mooneye demande un CPU découpé en M-cycles et un PPU plus fin. C'est un changement d'architecture, pas un correctif
- **WebAssembly** : compiler le cœur pour `wasm32-freestanding` et l'afficher dans un `<canvas>`. Le cœur séparé du frontend depuis la phase 0 rend ça simple
- **Save states** : sérialiser tout l'état de la machine en un fichier
- **Game Boy Color** : double vitesse, banques de VRAM et de WRAM, palettes couleur

Pièges connus et références

Pièges qui coûtent des soirées.

Piège	Phase	Ce qu'il faut savoir
`DAA`	2	Dépend de N, H et C ; implémente-le à partir des Pan Docs, pas de mémoire
Half-carry de `ADD SP,e` et `LD HL,SP+e`	2	H et C se calculent sur l'octet bas, avec `e` traité comme non signé
Bits bas de F	2	Toujours 0, y compris après `POP AF`
Délai de `EI`	3	IME ne passe à 1 qu'après l'instruction suivante
HALT bug	3	IME à 0 + interruption en attente : l'octet suivant est lu deux fois
Compteur de ligne de la fenêtre	4	Distinct de LY ; n'avance que si la fenêtre a été dessinée sur la ligne
Bits 6 et 7 de 0xFF00	5	Toujours lus à 1, sinon certains jeux lisent des touches fantômes
Banque ROM 0	6	Une écriture de 0 dans le registre de banque sélectionne la banque 1
Version de Zig	toutes	Exemples en ligne souvent périmés : vérifie la version visée

ROMs. Les ROMs de test sont libres. Pour Tetris et Pokémon Rouge, utilise des dumps de cartouches que tu possèdes.

Références.

- Pan Docs : la référence matérielle, à garder ouverte en permanence
- Table des opcodes : cycles et flags de chaque instruction
- Game Boy: Complete Technical Reference (Gekkio) : comportement précis du CPU, cycle par cycle
- SingleStepTests sm83 et Gameboy Doctor : tests du CPU
- ROMs de Blargg, mooneye-test-suite, dmg-acid2 : ROMs de test
- Ziglings et la documentation du langage : Zig